

PYTHON FOR DATA SCIENCE • MODULE 1 • TOPIC 1.15

Filtering, Selection & Indexing

Day 16 — loc, iloc, and the trap between them

90 min

Concepts + live code

30 min

Practice notebook

2

Words you must not mix up

Prepared & presented by **Srinivasa Sai Chava**



Boston University

Before we start — five questions from Day 15

1 **df["col"] or df[["col"]] — which is which?**

→ Series, then DataFrame.

2 **Why avoid df.count ?**

→ It is a method, not your column.

3 **NumPy mean with a nan vs pandas?**

→ nan • pandas skips it silently.

4 **Which ddof does pandas std use?**

→ ddof=1. NumPy uses 0.

5 **First three things on a new dataset?**

→ head(), shape, info().

Yesterday was about columns. Today is about ROWS — which is a great deal fussier.

How today's two hours are structured

1

Label vs position

loc and iloc, and the slicing difference

25 min

2

Boolean filtering

Masks on a DataFrame, & | ~

25 min

3

Handy filters

isin, between, query, sort_values

15 min

4

Setting the index

set_index, reset_index — and the trap

15 min

5

Mini build

Answer five questions about a dataset

5 min

6

Practice notebook

You code, I walk around

30 min

Sections 1 and 4 are marked in red because they cause the most confusion. They are also connected — the trap in 4 is caused by the distinction in 1.

By the end of today you will be able to

1

Choose

Say instantly whether a task needs loc or iloc

2

Filter

Select rows by condition, with several conditions combined

3

Assign

Change values in place without silently losing the change

4

Re-index

Set a meaningful index, and reset it when filtering breaks it

One sentence to carry through the whole session

loc speaks LABELS. iloc speaks POSITIONS. Every mistake today comes from asking one of them a question in the other's language — and because a DataFrame's labels often happen to BE 0, 1, 2, the two agree just often enough to lull you into not thinking about it.

1

Label vs Position

loc and iloc · 25 minutes

- The two questions
- Rows and columns
- The slicing difference

loc speaks labels. iloc speaks positions.

	name	city	python
0	Ravi	Pune	88
1	Sara	Mumbai	91
2	Amit	Delhi	45
3	Neha	Pune	67

labels ↑ positions are 0,1,2,3 as well — they MATCH here

```
df.loc[1, "name"] -> Sara    df.iloc[1, 0] -> Sara
```

Ask yourself one question

	loc	iloc
speaks	LABELS	POSITIONS
row 2 means	the row labelled 2	the 3rd row
[-1]	KeyError	the last row
columns by	name	number
with a mask	yes	no

If you are counting from the top, you want iloc. If you are naming something, you want loc.

⚠ They agree on a fresh DataFrame — which is the trap

A DataFrame straight out of `read_csv` has labels 0, 1, 2, 3 — exactly the same as its positions. So `loc` and `iloc` return the same thing, and you can write either one without ever learning the difference. Then you filter the rows, the labels stop matching the positions, and half your code quietly breaks. Section 4 shows exactly that.

The slicing difference — one includes, one does not

df.loc[1:3] — INCLUSIVE

	name	python
0	Ravi	88
1	Sara	91
2	Amit	45
3	Neha	67

three rows: 1, 2 AND 3

df.iloc[1:3] — EXCLUSIVE

	name	python
0	Ravi	88
1	Sara	91
2	Amit	45
3	Neha	67

two rows: 1 and 2 only

Why the inconsistency is not really an inconsistency

iloc slices by position, so it follows Python's rule from Day 4 — the stop is excluded, exactly like a list or range(). loc slices by label, and with labels there is no “next one” to stop before: if you ask for “Ravi to Neha” you clearly mean Neha included. Once you think of loc as naming a first and last row rather than counting, the behaviour is the obvious one.

2

Boolean Filtering

Selecting rows by condition · 25 minutes

- The mask
- Combining conditions
- loc for assignment

Filtering rows — Day 12's mask, on a table

```
mask = df["python"] > 70
```

```
0    True
1    True
2   False
3   False
```

```
df[mask]          # or, in one line:
df[df["python"] > 70]
```

```
   name  city  python
0  Ravi  Pune     88
1  Sara  Mumbai    91
```

A mask is a boolean Series with the same index. Passing it to `df` keeps only the rows where it is `True` — exactly Day 12's array masking, with labels attached.

Combining conditions

```
df[(df["python"] > 70) & (df["stats"] > 70)]
df[(df["city"] == "Pune") | (df["python"] > 85)]
df[~(df["city"] == "Pune")]
```

```
# & and | or ~ not
```

⚠ Exactly the Day 13 rules

```
... and ... -> ValueError: truth value
              of a Series is ambiguous
```

Use `&` and `|`, never `and` / `or`. And bracket every condition — `&` binds tighter than the comparisons, so leaving them off changes the meaning.

Changing values — and the assignment that vanishes

⚠ This does NOT work

```
df[df["python"] < 50]["python"] = 50

df.loc[2, "python"] -> 45

# still 45. Nothing changed.
```

Two brackets in a row means pandas built a temporary copy, wrote 50 into the copy, and threw the copy away. Modern pandas warns; older versions sometimes worked and sometimes did not.

This does — one set of brackets

```
df.loc[df["python"] < 50,
       "python"] = 50

df.loc[2, "python"] -> 50

# changed, as intended.
```

`loc` takes the rows AND the column in one operation, so pandas knows you are addressing the original.

The rule, and where you have met it before

To change data, use ONE `.loc[rows, column]` — never two square brackets in a row. This is the copy-versus-view question from Day 12 in a new costume: there, a NumPy slice was a view and surprised you by changing the original; here, a filtered DataFrame is a copy and surprises you by NOT changing it. Same underlying question, opposite symptom.

3

Handy Filters

Shorter ways to say common things · 15 minutes

- `isin`
- `between`
- `query`
- `sort_values`

Four shortcuts worth knowing

Instead of	Write	Note
<code>(c=="Pune") (c=="Delhi")</code>	<code>c.isin(["Pune", "Delhi"])</code>	scales to any number of values
<code>(p>=60) & (p<=90)</code>	<code>p.between(60, 90)</code>	INCLUSIVE at both ends
<code>df[(df.a>1) & (df.b<2)]</code>	<code>df.query("a > 1 and b < 2")</code>	here <code>and</code> is allowed
sorting by hand	<code>df.sort_values("python")</code>	add <code>ascending=False</code> to reverse

Top and bottom, quickly

```
df.nlargest(3, "python")
df.nsmallest(3, "python")

# sort_values(...).head(3), in one call
```

Both keep the whole row, not just the column you sorted on.

⚠ between includes both ends

```
p.between(60, 90)    keeps 60 and 90
p.between(60, 90, inclusive="left")
```

Unlike almost every other range in Python, both endpoints are kept by default. If a boundary value matters — a pass mark, an age limit — check which side you meant before you report the number.

4

Setting the Index

And the trap it creates · 15 minutes

- `set_index`
- The filtered-index trap
- `reset_index`

The trap — filtering breaks the label/position match

After a filter, labels have GAPS

```
top = df[df["python"] > 70]
```

	name	python
0	Ravi	88
1	Sara	91
4	Kiran	72

labels 0, 1, 4 — positions 0, 1, 2

Now they disagree

```
top.iloc[2]["name"] -> Kiran
# the 3rd row

top.loc[2]
-> KeyError: 2
# label 2 was Amit - filtered out!

top.loc[4]["name"] -> Kiran
# label 4 still exists
```

The fix: reset_index when the old labels no longer mean anything

```
top = top.reset_index(drop=True) -> 0, 1, 2
```

Without `drop=True` the old labels are kept as a new column called `index`. Use `drop=True` whenever the old numbering was meaningless — which, after a filter, it usually is.

Giving the rows meaningful names

```
df = df.set_index("name")
```

	city	python	stats
Ravi	Pune	88	71
Sara	Mumbai	91	84
Amit	Delhi	45	38

name is now the INDEX, not a column

Now loc reads beautifully

```
df.loc["Sara"]  
df.loc["Sara", "python"]    -> 91  
df.loc["Ravi":"Amit"]       three rows  
  
df.iloc[0]                   still Ravi  
# iloc NEVER changes - always positions
```

This is where loc's inclusive slicing feels obviously right: Ravi to Amit means Amit included.

When to set an index — and when not to bother

Set one when the rows have a natural identifier you will look things up by: a student ID, a product code, a date. Leave the default 0, 1, 2 when they do not — a made-up index adds nothing. Dates are the case where it pays off most, and Module 4's time series work assumes it.

Debug corner — eight selection bugs

You wrote	What happens	Fix
<code>df[mask]["col"] = 5</code>	silently does nothing	<code>df.loc[mask, "col"] = 5</code>
<code>top.loc[2]</code> after filtering	KeyError — label 2 was removed	<code>top.iloc[2]</code>
<code>df.loc[-1]</code>	KeyError — there is no label -1	<code>df.iloc[-1]</code>
<code>df.loc[1:3]</code> expecting 2 rows	gives 3 — loc includes the stop	<code>df.iloc[1:3]</code>
<code>df[cond1 and cond2]</code>	ValueError: truth value ambiguous	<code>df[cond1 & cond2]</code>
<code>df[df.a>1 & df.b<2]</code>	wrong rows — & binds tighter	bracket each condition
<code>p.between(60, 90)</code> as 61-89	both ends are INCLUDED	<code>inclusive="neither"</code>
<code>reset_index()</code> without drop	old labels become a column	<code>reset_index(drop=True)</code>

Rows 1 and 2 are the two that cost real time. All eight are in today's practice notebook.

Putting it together — five questions, five filters

```
import pandas as pd

df = pd.read_csv("class.csv")

# 1. Who scored above 70 in python?
top = df[df["python"] > 70]

# 2. Who is strong in BOTH subjects?
both = df[(df["python"] > 70) & (df["stats"] > 70)]

# 3. Who is from Pune or Delhi?
local = df[df["city"].isin(["Pune", "Delhi"])]

# 4. The three highest python marks, whole rows kept
best = df.nlargest(3, "python")

# 5. Give everyone below 50 a resit mark of 50
df.loc[df["python"] < 50, "python"] = 50      # ONE loc

# Tidy the filtered result before reporting it
top = top.reset_index(drop=True)
print(top[["name", "python"]])
```



A mask is a Series

Day 12, with labels attached



& not and

third appearance of this rule



isin

clearer than a chain of ==



nlargest

keeps the WHOLE row



One .loc to assign

two brackets would vanish



reset_index(drop=True)

labels match positions again



Original untouched

top is a copy, df still has 5 rows

Every question is one line. The skill is choosing the right filter, not writing clever code.

Practice session — 30 minutes



Predict the output

loc vs iloc, slicing, filtered labels

8 questions



Spot & fix the bug

Chained assignment, loc after filter

4 questions



Write the code

Filter, combine, assign, sort, re-index

5 tasks

Your notebook

Day16_Practice_
Questions.ipynb

- Say “label” or “position” first
- Print .index after filtering
- Raise a hand — I am walking around
- Solutions shared after class

Before writing loc or iloc, say out loud which one you mean: am I naming this row, or counting to it?

TODAY IN TWELVE LINES

Recap

- 1 loc speaks LABELS. iloc speaks POSITIONS.
- 2 On a fresh DataFrame they agree — that is the trap.
- 3 loc slicing INCLUDES the stop; iloc excludes it.
- 4 `df.loc[-1]` is a `KeyError`; `df.iloc[-1]` is the last row.
- 5 A mask is a boolean Series: `df[df["col"] > 70]`.
- 6 Use `&` | `~` and bracket every condition. Never `and` / `or`.
- 7 To ASSIGN, use one `df.loc[rows, col] = value`.
- 8 `df[mask]["col"] = v` writes to a copy and vanishes.
- 9 Filtering keeps the ORIGINAL labels — gaps appear.
- 10 So `loc[2]` can fail on a row you can see. Use `iloc`, or `reset`.
- 11 `reset_index(drop=True)` makes labels match positions again.
- 12 `isin`, `between` (inclusive!), `query`, `nlargest`, `sort_values`.

Homework

Filter a dataset, then show loc and iloc disagreeing on the same number · Write five questions about a CSV and answer each with one filter · Set a meaningful index on a dataset and look up three rows by name

Next class — Topic 1.16

GroupBy & aggregation: split the table into groups, compute a summary for each, and put the answers back together